What Makes a Steam Game Well Reviewed?

Price, indie status, genre and release year across 27,000 games

Ayman Salha

11/1/2026

Contents

1 Background and Questions **2**

1.1 The data . . . 2

2 Data Wrangling and Cleaning **3**

2.1 Loading and first checks . . . 3

2.2 Building new variables . . . 4

2.3 Why a minimum number of reviews? . . . 5

2.4 Cleaning steps and how many games remain . . . 6

2.5 Joining the player-tag data . . . 7

3 Exploratory Data Analysis **8**

3.1 An overview . . . 8

3.2 Question 1: Are indie games reviewed better? . . . 10

3.3 Question 2: Is there a price sweet spot? . . . 11

3.4 Question 3: Which kinds of games are well liked? . . . 13

3.5 Question 4: Did the flood of releases lower review scores? . . . 16

3.6 Putting it together: multiple regression . . . 19

4 Conclusions **21**

4.1 Limitations . . . 21

1 Background and Questions

Steam is the largest PC game store, with tens of thousands of games. Each game has a public review score: players either recommend it (positive) or not (negative). For developers, a good review score matters because Steam uses it to decide which games to show, and players use it to decide what to buy.

This project asks what kinds of games get better reviews. Specifically:

1. **Indie games:** Are games labeled *Indie* reviewed better or worse than games from larger studios?
2. **Price:** Is there a price "sweet spot"? Do cheaper games get better or worse reviews?
3. **Genres and tags:** Which kinds of games are popular to make but poorly reviewed, and which are well liked?
4. **Release year:** Steam opened up to many more developers over time (Steam Greenlight in 2012, then Steam Direct in 2017). Did the flood of new releases come with lower review scores?

1.1 The data

The data is the **Steam Store Games (Clean dataset)** by Nik Davis, published on Kaggle (https://www.kaggle.com/datasets/nikdavis/steam-store-games). It was collected from the Steam store and the SteamSpy API in **May 2019**, so it does not include newer games. I use two of its files:

- `steam.csv`: one row per game (27,075 games) with these variables:

Variable	Meaning
`appid`	Steam's unique ID for the game
`name`, `developer`, `publisher`	Game title and who made / sold it
`release_date`	Release date (YYYY-MM-DD)
`english`	1 if the game supports English
`platforms`	Supported systems, e.g. `windows;mac;linux`
`required_age`	Minimum age (0 = none)
`categories`	Store features, e.g. `Single-player;Steam Achievements`
`genres`	Store genres, semicolon-separated, e.g. `Action;Indie`
`steamspy_tags`	Top 3 player-applied tags
`achievements`	Number of Steam achievements
`positive_ratings`, `negative_ratings`	Counts of positive and negative reviews
`average_playtime`, `median_playtime`	Playtime in minutes
`owners`	Estimated owners, as a *range of text* such as `20000-50000`
`price`	Price in GBP at collection time (0 = free)

- `steamspy_tag_data.csv`: one row per game and one column for each of 371 player tags (for example `pixel_graphics`, `survival`), holding how many players applied that tag to the game.

2 Data Wrangling and Cleaning

2.1 Loading and first checks

```
steam_raw <- read.csv("steam.csv", stringsAsFactors = FALSE)
tags_raw  <- read.csv("steamspy_tag_data.csv", check.names = FALSE)

dim(steam_raw)
```

```
## [1] 27075    18
```

```
dim(tags_raw)
```

```
## [1] 29022   372
```

```
# Missing values and empty strings in the main table
sum(is.na(steam_raw))
```

```
## [1] 0
```

```
sapply(steam_raw[, c("developer", "publisher", "genres", "platforms")],
       function(x) sum(x == ""))
```

```
## developer publisher    genres platforms
##         0         0         0         0
```

There are no missing values, so cleaning is mostly about **turning text fields into usable variables** and **removing rows that would distort the analysis**.

```
# Are there duplicate games?
sum(duplicated(steam_raw$appid))
```

```
## [1] 0
```

```
dup_names <- steam_raw %>%
  group_by(name) %>% filter(n() > 1) %>% ungroup()
nrow(dup_names)
```

```
## [1] 83
```

```r
dup_names %>% arrange(name) %>%
  select(appid, name, release_date, developer) %>% head(6) %>% kable()
```

appid	name	release_date	developer
942050	2048	2018-10-31	Lucas Alexander Visual Arts;Nicolai Clemmensen;Gabriele Cirulli
1026330	2048	2019-02-21	Dexion Games
837930	Alone	2018-05-01	Killed Pixel Games
871870	Alone	2018-06-21	Avasion
63110	Alter Ego	2010-08-03	bitComposer Games
664780	Alter Ego	2017-08-08	Choose Multiple LLC

No `appid` repeats. Some **names** repeat, but they are different games by different developers that share a title (for example two games called *Alone*), so they are kept.

2.2 Building new variables

```r
non_game_genres <- c("Utilities", "Design & Illustration", "Animation & Modeling",
                     "Education", "Video Production", "Software Training",
                     "Audio Production", "Web Publishing", "Game Development",
                     "Photo Editing", "Accounting", "Documentary", "Tutorial")

steam <- steam_raw %>%
  mutate(
    # Reviews: total count and share positive (the review score)
    n_reviews = positive_ratings + negative_ratings,
    score     = positive_ratings / n_reviews,

    # Dates: text -> Date, plus the release year
    release_date = as.Date(release_date),
    year         = as.integer(format(release_date, "%Y")),

    # Owners: "20000-50000" -> numeric lower bound, upper bound and midpoint
    owners_low  = as.numeric(sub("-.*", "", owners)),
    owners_high = as.numeric(sub(".*-", "", owners)),
    owners_mid  = (owners_low + owners_high) / 2,

    # Genres: flags for the groups we compare
    indie        = grepl("Indie", genres),
    early_access = grepl("Early Access", genres),
    free         = price == 0,
```

```
    # Software (e.g. video editors) is sold on Steam too. A row is a game if
    # at least one of its genres is not a software genre.
    is_game = sapply(strsplit(genres, ";"),
                     function(g) any(!g %in% non_game_genres)),

    # Price groups, in pounds
    price_group = cut(price,
                      breaks = c(-Inf, 0, 4.99, 9.99, 19.99, Inf),
                      labels = c("Free", "Under 5", "5-10", "10-20", "20+"))
  )

steam %>%
  select(name, owners, owners_mid, indie, free, price_group, score) %>%
  head(4) %>%
  kable(digits = 3, format.args = list(big.mark = ",", scientific = FALSE))
```

name	owners	owners_mid	indie	free	price_group	score
Counter-Strike	10000000-20000000	15,000,000	FALSE	FALSE	5-10	0.974
Team Fortress Classic	5000000-10000000	7,500,000	FALSE	FALSE	Under 5	0.840
Day of Defeat	5000000-10000000	7,500,000	FALSE	FALSE	Under 5	0.896
Deathmatch Classic	5000000-10000000	7,500,000	FALSE	FALSE	Under 5	0.827

2.3 Why a minimum number of reviews?

A game with only 3 reviews can easily score 0% or 100% by chance. The plot below shows that games with few reviews have wildly spread-out scores, and the spread narrows as the number of reviews grows.

```
ggplot(steam, aes(n_reviews, score)) +
  geom_point(alpha = 0.08, size = 0.6, colour = steam_blue) +
  geom_vline(xintercept = 50, linetype = "dashed", colour = accent) +
  scale_x_log10(labels = comma) +
  scale_y_continuous(labels = percent) +
  labs(x = "Number of reviews (log scale)", y = "Positive reviews",
       title = "Small games have unreliable scores")
```

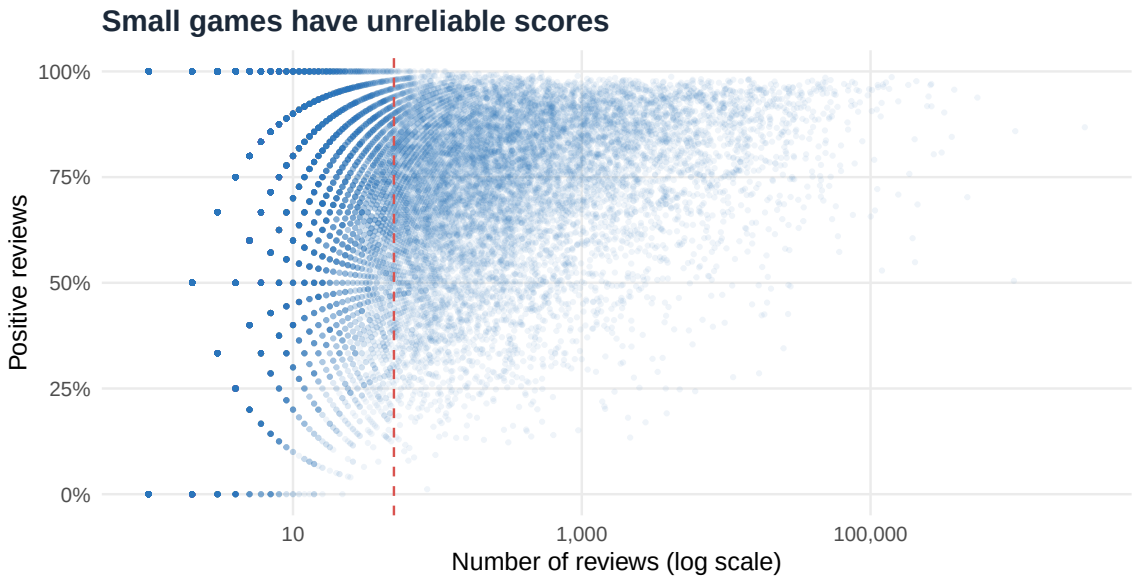


Figure 1: Review score vs. number of reviews (log scale). Scores are unreliable for games with few reviews; the dashed line marks the 50-review cutoff.

I therefore keep only games with **at least 50 reviews**. This also removes most games that almost nobody played.

2.4 Cleaning steps and how many games remain

```
steam_2019_cutoff <- max(steam$release_date)   # data collected in May 2019

games <- steam %>%
  filter(is_game) %>%            # 1. drop software that is not a game
  filter(english == 1) %>%       # 2. English-language store pages only
  filter(n_reviews >= 50)        # 3. enough reviews for a reliable score

steps <- data.frame(
  Step  = c("Raw data", "Games only (no software)", "English only",
            "At least 50 reviews"),
  Games = c(nrow(steam),
            sum(steam$is_game),
            sum(steam$is_game & steam$english == 1),
            nrow(games))
)
kable(steps, format.args = list(big.mark = ","))
```

Step	Games
Raw data	27,075

Step	Games
Games only (no software)	26,919
English only	26,409
At least 50 reviews	11,560

```
overall_mean <- mean(games$score)    # used as a reference line in later plots
```

2.5 Joining the player-tag data

Store genres are broad (most games are "Action", "Indie" or "Casual"). Player tags are much more specific. The tag file is **wide**: one column per tag. I reshape it to **long** form (one row per game-tag pair), keep tags that make up at least 5% of a game's tag votes (so a single stray vote does not count), and join it to the cleaned games.

```
# Games in the tag file that have no store data
sum(!tags_raw$appid %in% steam_raw$appid)
```

```
## [1] 1947
```

```
game_tags <- tags_raw %>%
  pivot_longer(-appid, names_to = "tag", values_to = "votes") %>%
  filter(votes > 0) %>%
  group_by(appid) %>%
  mutate(share = votes / sum(votes)) %>%
  ungroup() %>%
  filter(share >= 0.05) %>%
  inner_join(games %>% select(appid, score), by = "appid") %>%
  mutate(tag = gsub("_", " ", tag))

nrow(game_tags)
```

```
## [1] 77808
```

```
head(game_tags, 4) %>% kable(digits = 3)
```

appid	tag	votes	share	score
10	action	2681	0.172	0.974
10	classic	1344	0.086	0.974
10	competitive	790	0.051	0.974
10	fps	2048	0.131	0.974

The tag file lists 1,947 apps that do not appear in `steam.csv`, so an `inner_join` keeps only games present in both tables.

3 Exploratory Data Analysis

3.1 An overview

```
ggplot(games, aes(score)) +
  geom_histogram(binwidth = 0.025, fill = steam_blue, colour = "white") +
  geom_vline(xintercept = median(games$score), linetype = "dashed") +
  scale_x_continuous(labels = percent) +
  labs(x = "Positive reviews", y = "Number of games",
       title = "Most games are reviewed positively")
```

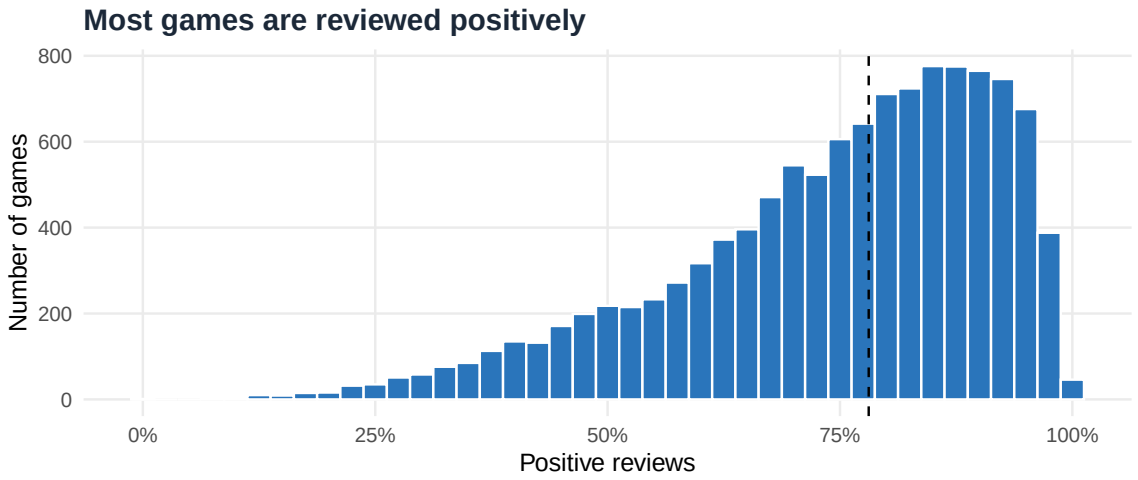


Figure 2: Distribution of review scores for the cleaned games.

```
summary(games$score)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.01176 0.64564 0.78090 0.74404 0.87911 1.00000
```

The typical game has about 78% positive reviews, and the distribution is skewed: there is a long tail of poorly reviewed games. Because of that skew I report medians as well as means, and the large sample sizes make the t-tests and ANOVA below reliable even though the scores are not normally distributed (Central Limit Theorem).

3.1.1 Popularity and review score

The `owners` ranges cleaned earlier let us check whether more popular games are also better reviewed. Each range becomes one ordered group:

```
short_num <- function(x) ifelse(x >= 1e6, paste0(x / 1e6, "M"), paste0(x / 1e3, "k"))

owner_groups <- games %>%
  mutate(owner_group = ifelse(owners_low >= 2000000, "2M+",
                              paste0(short_num(owners_low), "-",
                                     short_num(owners_high))),
         owner_group = reorder(owner_group, owners_low))

ggplot(owner_groups, aes(owner_group, score)) +
  geom_boxplot(fill = steam_light, alpha = 0.6, outlier.alpha = 0.1) +
  scale_y_continuous(labels = percent) +
  labs(x = "Estimated owners", y = "Positive reviews",
       title = "More popular games tend to be better reviewed")
```

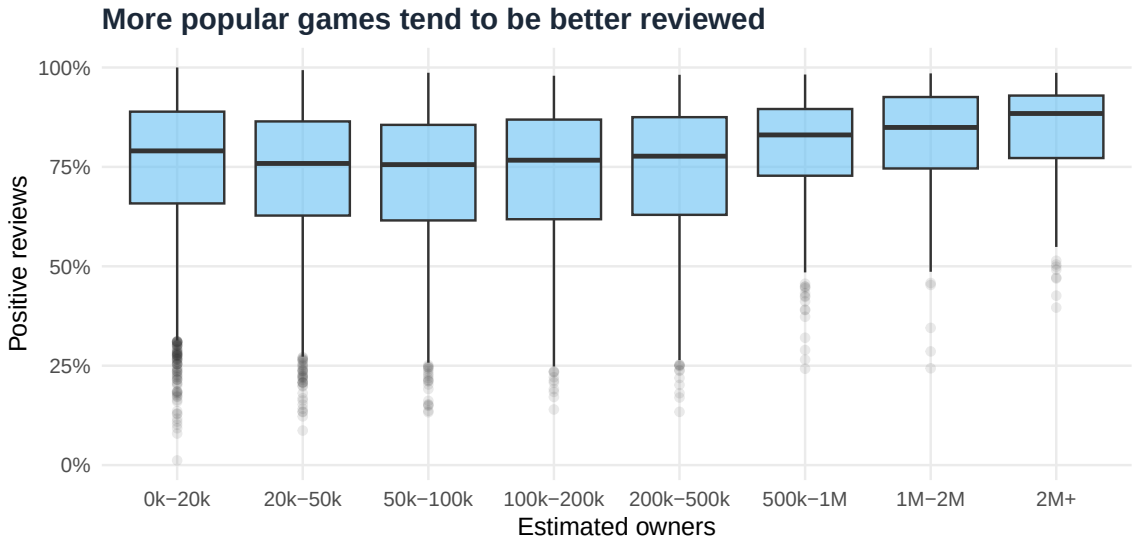


Figure 3: Review score by estimated number of owners. All ranges of 2 million or more are combined into one group.

```
owner_groups %>%
  group_by(owner_group) %>%
  summarise(games = n(), median_score = median(score)) %>%
  kable(digits = 3)
```

owner_group	games	median_score
0k-20k	3868	0.790
20k-50k	2450	0.759
50k-100k	1580	0.756
100k-200k	1344	0.767
200k-500k	1260	0.777

owner_group	games	median_score
500k-1M	509	0.830
1M-2M	285	0.849
2M+	264	0.884

From about 50,000 owners upward, median scores rise steadily: the most-owned games have a median of about 88% positive. Popularity and quality go together, though the direction is unclear: good games attract players, and popular games may also attract more forgiving fans.

The smallest group (under 20,000 owners) is the exception, with a fairly high median. These are small games that still collected 50+ reviews, which usually means a small but enthusiastic audience, such as niche visual novels or puzzle games.

3.2 Question 1: Are indie games reviewed better?

```
games <- games %>%
  mutate(indie_label = ifelse(indie, "Indie", "Not indie"))

ggplot(games, aes(indie_label, score, fill = indie_label)) +
  geom_violin(alpha = 0.5, colour = NA) +
  geom_boxplot(width = 0.15, outlier.shape = NA) +
  stat_summary(fun = mean, geom = "point", shape = 23, size = 3, fill = "white") +
  scale_fill_manual(values = c("Indie" = steam_light, "Not indie" = steam_dark)) +
  scale_y_continuous(labels = percent) +
  labs(x = NULL, y = "Positive reviews", title = "Indie vs. not indie") +
  theme(legend.position = "none")
```

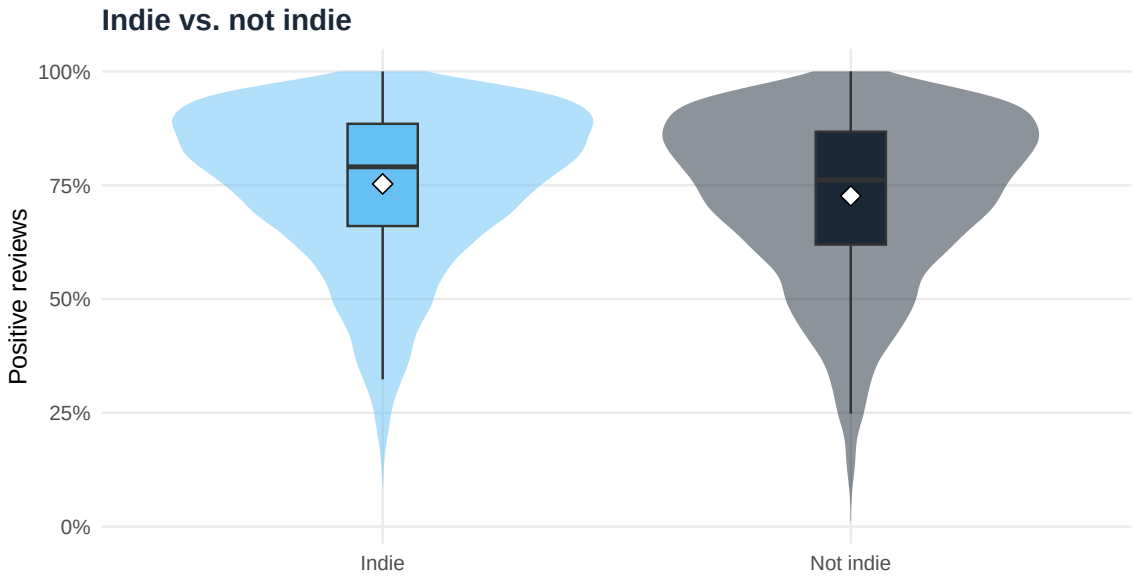


Figure 4: Review scores for indie and non-indie games. The white diamond marks the mean.

```r
games %>%
  group_by(indie_label) %>%
  summarise(games = n(), mean = mean(score), median = median(score)) %>%
  kable(digits = 3)
```

indie_label	games	mean	median
Indie	7727	0.753	0.790
Not indie	3833	0.726	0.762

```r
indie_test <- t.test(score ~ indie, data = games)   # Welch two-sample t-test
indie_test
```

```
##
##  Welch Two Sample t-test
##
## data:  score by indie
## t = -7.6321, df = 7275.2, p-value = 2.603e-14
## alternative hypothesis: true difference in means between group FALSE and group TRUE is not e
## 95 percent confidence interval:
##  -0.03318589 -0.01962228
## sample estimates:
## mean in group FALSE  mean in group TRUE
##           0.7263925           0.7527966
```

Hypotheses. $H_0 : \mu_{\text{indie}} = \mu_{\text{not indie}}$ vs. $H_a : \mu_{\text{indie}} \neq \mu_{\text{not indie}}$.

Result. Indie games average about 2.6 percentage points more positive reviews, and the p-value is tiny, so we reject H_0. The difference is real but **small**: both groups are mostly positive, and the boxes overlap heavily. Being indie is not a guarantee of quality, but indie games are not worse on average, despite usually having much smaller budgets.

3.3 Question 2: Is there a price sweet spot?

```r
ggplot(games, aes(price_group, score)) +
  geom_boxplot(fill = steam_light, alpha = 0.6, outlier.alpha = 0.15) +
  stat_summary(fun = mean, geom = "point", shape = 23, size = 3, fill = "white") +
  scale_y_continuous(labels = percent) +
  labs(x = "Price (GBP)", y = "Positive reviews",
       title = "Free and cheap games are reviewed worst")
```

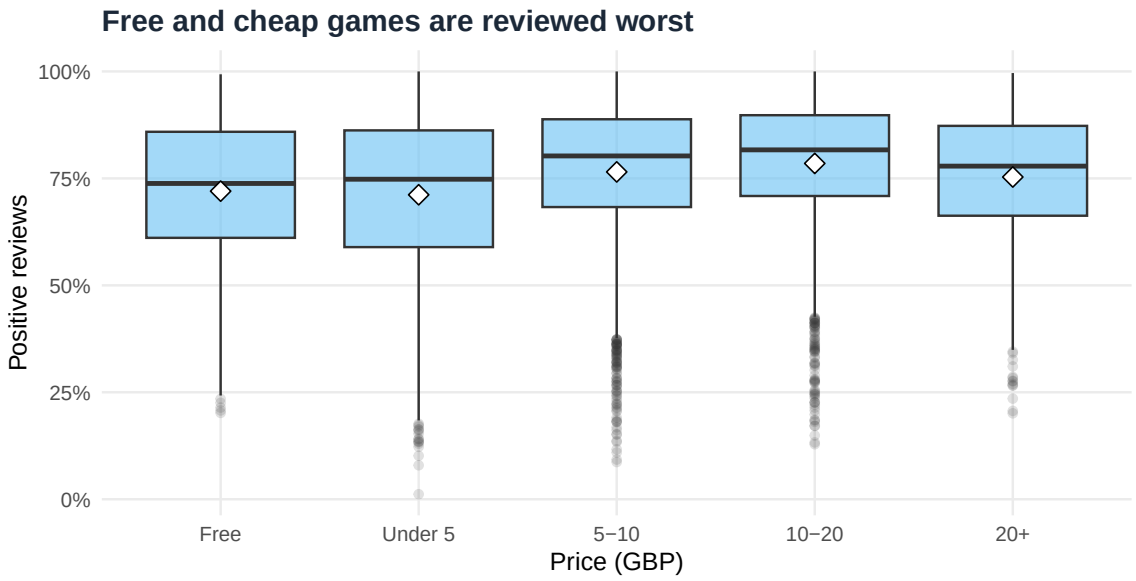


Figure 5: Review scores by price group (GBP). The white diamond marks the mean.

```
games %>%
  group_by(price_group) %>%
  summarise(games = n(), mean = mean(score), median = median(score)) %>%
  kable(digits = 3)
```

price_group	games	mean	median
Free	1508	0.720	0.738
Under 5	4068	0.712	0.748
5-10	2718	0.765	0.802
10-20	2535	0.785	0.817
20+	731	0.753	0.779

```
price_aov <- aov(score ~ price_group, data = games)
summary(price_aov)
```

```
##                Df Sum Sq Mean Sq F value Pr(>F)
## price_group     4   10.6  2.6600   92.45 <2e-16 ***
## Residuals   11555  332.5  0.0288
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The one-way ANOVA tests H_0: all five price groups have the same mean score. The p-value is far below 0.05, so at least one group differs. To see **which** groups differ, Tukey's test compares every pair while controlling the overall error rate:

```r
tukey <- TukeyHSD(price_aov)$price_group
tukey_df <- data.frame(Comparison = rownames(tukey),
                       Difference = tukey[, "diff"],
                       `Adj. p-value` = tukey[, "p adj"],
                       check.names = FALSE, row.names = NULL)
kable(tukey_df, digits = 4)
```

Comparison	Difference	Adj. p-value
Under 5-Free	-0.0085	0.4568
5-10-Free	0.0449	0.0000
10-20-Free	0.0648	0.0000
20+-Free	0.0330	0.0002
5-10-Under 5	0.0535	0.0000
10-20-Under 5	0.0733	0.0000
20+-Under 5	0.0415	0.0000
10-20-5-10	0.0199	0.0002
20+-5-10	-0.0119	0.4425
20+-10-20	-0.0318	0.0001

Free games and games priced **under 5** have the lowest scores. Tukey's test finds no significant difference between these two groups (p is about 0.46), but both score significantly lower than games priced 5-10 or 10-20. Games priced **10-20** have the highest scores, and games priced **20+** score significantly *lower* than them. So there is a sweet spot in the middle rather than "more expensive is always better." For free games, a likely reason is that many earn money through in-game purchases, which players often review negatively.

3.4 Question 3: Which kinds of games are well liked?

3.4.1 Store genres

```r
genre_stats <- games %>%
  select(appid, genres, score) %>%
  mutate(genre = strsplit(genres, ";")) %>%
  unnest(genre) %>%
  filter(!genre %in% non_game_genres) %>%
  group_by(genre) %>%
  summarise(n_games = n(), mean_score = mean(score)) %>%
  filter(n_games >= 100)

kable(arrange(genre_stats, mean_score), digits = 3)
```

genre	n_games	mean_score
Massively Multiplayer	485	0.634
Gore	196	0.684
Racing	460	0.700
Violent	281	0.701
Simulation	2358	0.707
Sports	523	0.707
Free to Play	1185	0.709
Early Access	978	0.721
Strategy	2500	0.725
Action	5250	0.736
Casual	3257	0.746
RPG	2212	0.746
Adventure	4451	0.750
Indie	7727	0.753
Nudity	138	0.767
Sexual Content	128	0.797

```
ggplot(genre_stats, aes(mean_score, reorder(genre, mean_score))) +
  geom_vline(xintercept = overall_mean, linetype = "dashed", colour = "grey50") +
  geom_segment(aes(x = overall_mean, xend = mean_score, yend = genre),
               colour = "grey70") +
  geom_point(aes(size = n_games,
                 colour = mean_score > overall_mean)) +
  scale_colour_manual(values = c("TRUE" = steam_blue, "FALSE" = accent),
                      guide = "none") +
  scale_size(range = c(1.5, 6), labels = comma, name = "Games") +
  scale_x_continuous(labels = percent) +
  labs(x = "Average positive reviews", y = NULL,
       title = "Which store genres are reviewed best?",
       caption = "Dashed line = average over all games")
```

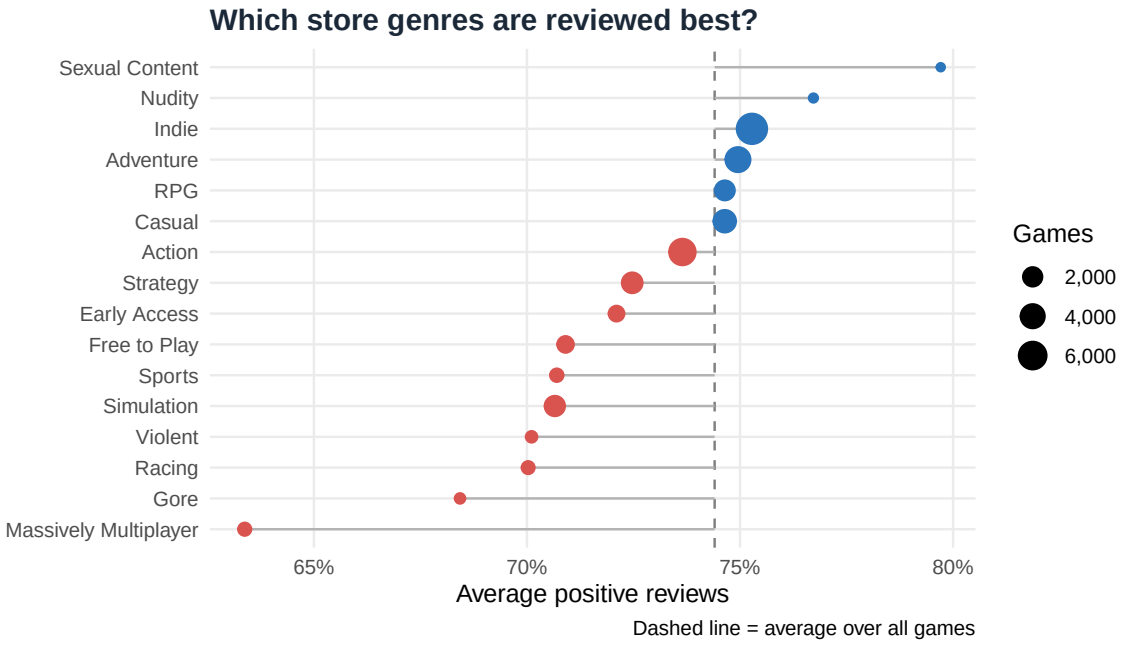


Figure 6: Average review score for each store genre (genres on at least 100 games). Point size shows how many games use the genre.

Massively Multiplayer games are the clear outlier at the bottom. They depend on servers, a large player base and ongoing updates, which gives players many reasons to leave a negative review. Racing, Sports and Simulation are also below average. Adventure, Casual and Indie sit above average.

3.4.2 Player tags

```
tag_stats <- game_tags %>%
  group_by(tag) %>%
  summarise(n_games = n(), mean_score = mean(score)) %>%
  filter(n_games >= 150)

tag_extremes <- bind_rows(
  tag_stats %>% slice_min(mean_score, n = 10) %>% mutate(group = "Worst 10"),
  tag_stats %>% slice_max(mean_score, n = 10) %>% mutate(group = "Best 10")
)

ggplot(tag_extremes, aes(mean_score, reorder(tag, mean_score), colour = group)) +
  geom_vline(xintercept = overall_mean, linetype = "dashed", colour = "grey50") +
  geom_segment(aes(x = overall_mean, xend = mean_score, yend = tag)) +
  geom_point(size = 3) +
  scale_colour_manual(values = c("Best 10" = steam_blue, "Worst 10" = accent)) +
  scale_x_continuous(labels = percent) +
  labs(x = "Average positive reviews", y = NULL, colour = NULL,
```

```
       title = "Best and worst player tags") +
  theme(legend.position = "top")
```

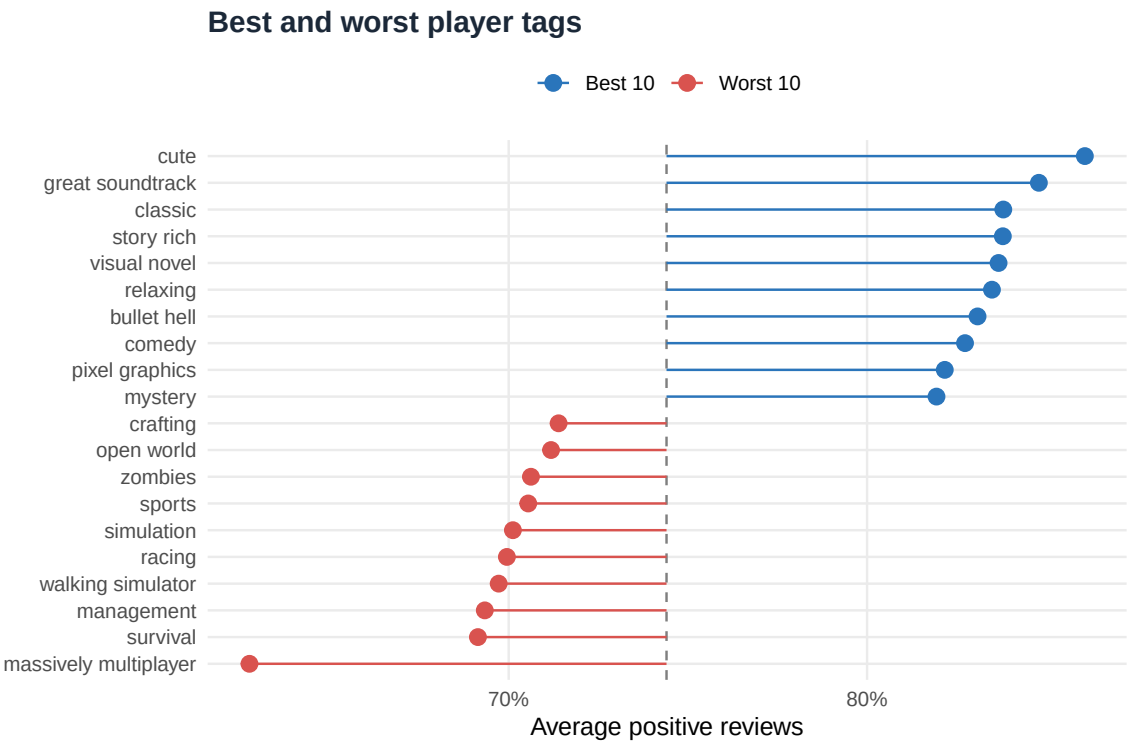


Figure 7: The 10 best- and 10 worst-reviewed player tags (tags on at least 150 games).

The worst tags describe **ambitious systems**: survival, crafting, open world, management, massively multiplayer. These games are hard to finish and balance, and many are sold unfinished. The best tags describe **smaller, focused experiences**: visual novels, pixel graphics, story-rich, cute.

One caution: players add tags *after* playing. Tags like *great soundtrack* or *classic* are partly praise, so their high scores are not purely caused by the kind of game. Tags describing what a game *is* (visual novel, survival, racing) are more trustworthy for this question.

3.5 Question 4: Did the flood of releases lower review scores?

```
by_year <- games %>%
  filter(year >= 2006) %>%
  group_by(year) %>%
  summarise(n_games = n(), mean_score = mean(score))

all_releases <- steam %>%
  filter(is_game, year >= 2006) %>%
  count(year, name = "released")
```

```
ggplot(all_releases, aes(year, released)) +
  geom_col(fill = steam_dark) +
  geom_vline(xintercept = c(2012.6, 2017.4), linetype = "dashed", colour = accent) +
  annotate("text", x = c(2012.6, 2017.4), y = max(all_releases$released),
           label = c("Greenlight", "Steam Direct"), hjust = 1.05, size = 3,
           colour = accent) +
  scale_y_continuous(labels = comma) +
  labs(x = "Release year", y = "Games released", title = "The release flood")
```

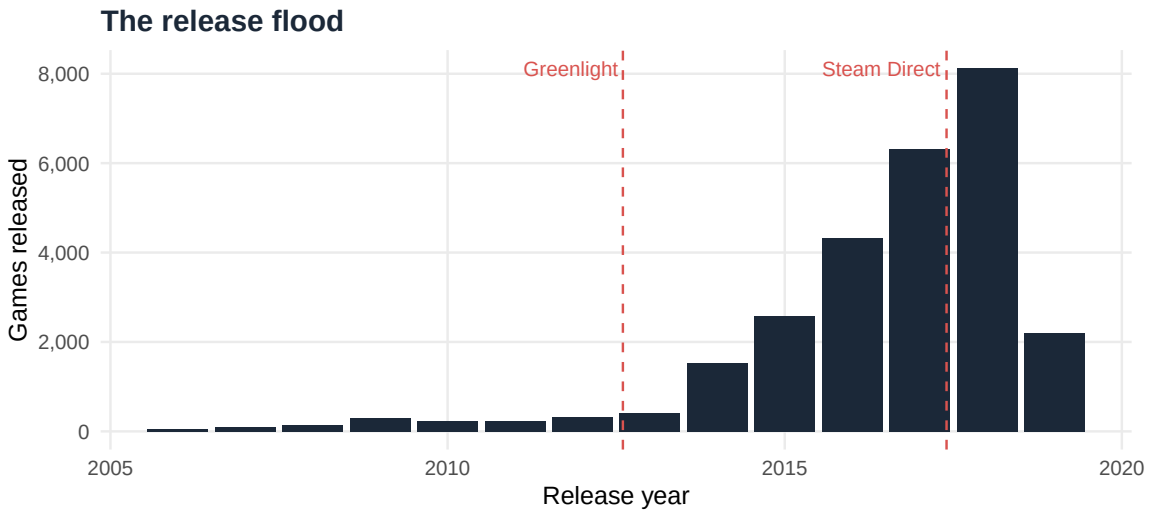


Figure 8: Games released each year (all games, before the review filter). 2019 covers January to May only.

```
ggplot(by_year, aes(year, mean_score)) +
  geom_line(colour = steam_blue, linewidth = 1) +
  geom_point(aes(size = n_games), colour = steam_blue) +
  scale_y_continuous(labels = percent) +
  scale_size(range = c(1, 4), guide = "none") +
  labs(x = "Release year", y = "Average positive reviews",
       title = "Average review score by release year")
```

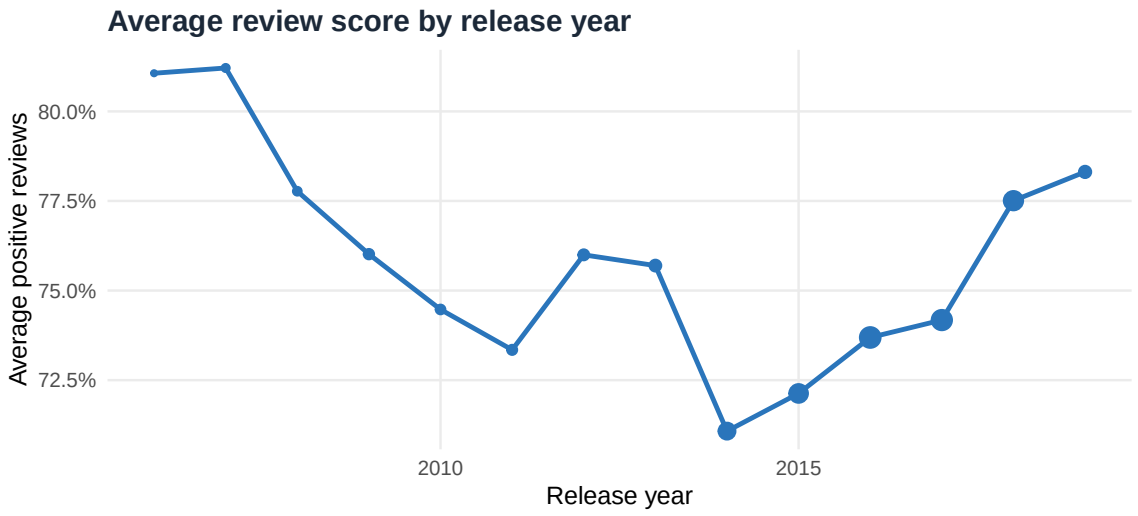


Figure 9: Average review score by release year (games with 50+ reviews). Point size shows how many games.

```
kable(by_year, digits = 3)
```

year	n_games	mean_score
2006	45	0.811
2007	87	0.812
2008	120	0.778
2009	241	0.760
2010	185	0.745
2011	216	0.733
2012	298	0.760
2013	398	0.757
2014	1300	0.711
2015	1741	0.721
2016	2330	0.737
2017	2208	0.742
2018	1916	0.775
2019	451	0.783

Releases exploded after 2013: Steam Greenlight (2012) and then Steam Direct (2017) made it much easier for small developers to publish. The average score **dropped to its lowest point in 2014**, the first big year of the flood, then recovered steadily through 2018 and 2019.

There is a catch called **survivorship bias**. Recent games had less time to collect 50 reviews, so the recent games that pass the filter tend to be the ones that became popular, and popular games tend to be good. Older games, by contrast, had years to collect reviews from everyone. So the recovery after 2014 may be partly real and partly a side effect of the filter.

3.6 Putting it together: multiple regression

Each question above looked at one factor at a time, but the factors overlap (for example, indie games tend to be cheaper). A multiple regression estimates each factor's effect **while holding the others constant**:

$$\text{score} = \beta_0 + \beta_1\,\text{indie} + \beta_2\,\text{price group} + \beta_3\,\log(\text{reviews}) + \beta_4\,\text{year} + \beta_5\,\text{early access} + \varepsilon$$

```
model <- lm(score ~ indie + price_group + log(n_reviews) + year + early_access,
            data = games)
summary(model)
```

```
##
## Call:
## lm(formula = score ~ indie + price_group + log(n_reviews) + year +
##     early_access, data = games)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.66025 -0.09304  0.02919  0.11870  0.35032
##
## Coefficients:
##                       Estimate Std. Error t value Pr(>|t|)
## (Intercept)        -9.9090918  1.4079834  -7.038 2.06e-12 ***
## indieTRUE           0.0378949  0.0035720  10.609  < 2e-16 ***
## price_groupUnder 5  0.0092481  0.0051999   1.779   0.0753 .
## price_group5-10     0.0589652  0.0054708  10.778  < 2e-16 ***
## price_group10-20    0.0613359  0.0053516  11.461  < 2e-16 ***
## price_group20+      0.0169734  0.0076132   2.229   0.0258 *
## log(n_reviews)      0.0267939  0.0010817  24.771  < 2e-16 ***
## year                0.0051815  0.0006976   7.427 1.19e-13 ***
## early_accessTRUE   -0.0412468  0.0056298  -7.327 2.52e-13 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.1641 on 11551 degrees of freedom
## Multiple R-squared:  0.09288,    Adjusted R-squared:  0.09225
## F-statistic: 147.8 on 8 and 11551 DF,  p-value: < 2.2e-16
```

```
coefs <- data.frame(term = names(coef(model)), est = coef(model),
                    confint(model), row.names = NULL) %>%
  filter(term != "(Intercept)") %>%
  mutate(term = recode(term,
           indieTRUE = "Indie", early_accessTRUE = "Early Access",
           `price_groupUnder 5` = "Price under 5 (vs free)",
           `price_group5-10` = "Price 5-10 (vs free)",
```

```
          `price_group10-20` = "Price 10-20 (vs free)",
          `price_group20+` = "Price 20+ (vs free)",
          `log(n_reviews)` = "log(number of reviews)",
          year = "Release year (+1)"))

ggplot(coefs, aes(est * 100, reorder(term, est))) +
  geom_vline(xintercept = 0, colour = "grey50") +
  geom_errorbar(aes(xmin = X2.5.. * 100, xmax = X97.5.. * 100),
                width = 0.2, orientation = "y") +
  geom_point(size = 3, colour = steam_blue) +
  labs(x = "Effect on positive reviews (percentage points)", y = NULL,
       title = "What predicts a better review score?")
```

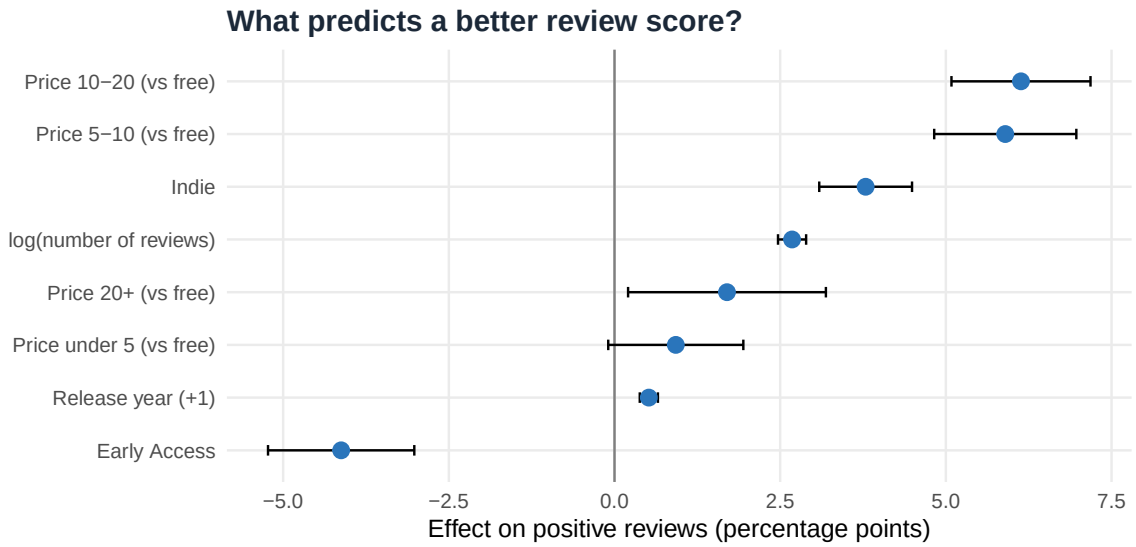


Figure 10: Regression estimates with 95% confidence intervals, in percentage points of positive reviews.

Holding the other factors fixed:

- **Indie** games still score about 3.8 points higher.
- Compared with free games, games priced **10-20** score about 6.1 points higher, the largest price effect.
- **Early Access** (unfinished) games score about 4.1 points lower.
- Games with **more reviews** score higher: popularity and quality go together.

The model's R^2 is only 0.093: these factors explain about 9% of the variation in scores. The effects are statistically clear, but most of what makes a game well-reviewed (how fun it is, how well it runs) is not in this dataset.

4 Conclusions

1. **Indie games are reviewed slightly better** than non-indie games, even after accounting for price and other factors. The difference is small but statistically significant.
2. **There is a price sweet spot.** Games in the 10-20 range are reviewed best. Free games and games under 5 are reviewed worst (about equally), and games over 20 score lower than the 10-20 group.
3. **Ambitious genres disappoint more often.** Massively multiplayer, survival, crafting and open-world games score lowest; focused, story-driven and visual-novel games score highest.
4. **The release flood lowered scores at first.** Average scores hit their low point in 2014 and then recovered, though survivorship bias likely exaggerates the recovery.
5. **Significant does not mean large.** With more than 10,000 games, even small differences produce tiny p-values. Effect sizes and the low R^2 show that none of these factors comes close to deciding a game's success on its own.

4.1 Limitations

- The data stops in **May 2019** and prices are in GBP at collection time.
- The **50-review filter** removes most small games, so conclusions apply to games that found at least a modest audience.
- Review score measures whether players recommend a game, not sales or quality directly, and **owner counts are rough ranges**.
- Tags are applied by players after playing, so some tags reflect opinion.
- This is observational data: these are associations, not causes.